

Hackathon SQL - DQLab

Discovering Sales Performance

Outlier Detection in Sales Orders
using SQL Statistical Methods

Ahmad Miftahul Farohi

Hackathon DQLab Hackathon 2026



Latar Belakang



Perusahaan

PT XYZ adalah distributor makanan kering yang memiliki jaringan sales lapangan tersebar secara hierarkis.



Kondisi Saat Ini

Purchase order terlihat baik di angka, namun banyak penjualan aktual yang mandek di lapangan.

Pemesanan yang tidak merata diduga menjadi penyebab utama ketimpangan antara PO dan realisasi penjualan.

Tantangan



Hierarki Kompleks

Struktur organisasi sales berbentuk tree multi-level, sulit dilacak secara manual.



Anomali Tersembunyi

Outlier tidak terlihat jelas dari angka agregat — butuh metode statistik yang tepat.



Batasan MySQL 5.7

Tidak bisa menggunakan Window Function, CTE, atau rekursi — hanya Temporary Table & JOIN.

Pertanyaan Utama



Bagaimana cara mengidentifikasi transaksi outlier dari sales lapangan berdasarkan Sales Manager Level 2 dengan SQL di MySQL 5.7?

- Bagaimana memetakan setiap transaksi ke Sales Manager Level 2?
- Bagaimana menghitung average & stddev per kelompok tanpa window function?
- Bagaimana menampilkan summary + detail outlier dalam satu query output?

Pendekatan Solusi

1



Mapping Hierarki

Traversal tree via self-JOIN berantai (10 level) → tentukan Level 2 tiap node

2



Mapping Orders

JOIN orders ke hasil hierarki → setiap order terhubung ke Level 2 manager-nya

3



Statistik per Group

GROUP BY level2 → hitung AVG & STDDEV_POP per kelompok manager

4



Deteksi Outlier

Filter: $\text{nilai_order} > \text{avg} + 3\sigma$ ATAU $\text{nilai_order} < \text{avg} - 3\sigma$ → hitung Z-score

5



Summary + Detail

UNION ALL: baris ringkasan jumlah anomali + baris detail tiap outlier

SQL Solution Walkthrough



6 Temporary Tables → 1 Final Output

tmp_hierarchy

Self-JOIN 10x → level2 tiap node

tmp_orders_mapped

JOIN orders → peta ke level2

tmp_stats

AVG & STDDEV_POP per level2

tmp_outliers

Filter $\pm 3\sigma$ + hitung Z-score

tmp_summary

COUNT anomali per level2

Final UNION ALL

Summary rows + detail rows

Key Technical Highlights

Self-JOIN Berlapis (bukan Rekursi)

MySQL 5.7 tidak support rekursi. Solusi: JOIN tabel nodes dengan dirinya sendiri hingga 10 kali untuk menelusuri jalur dari leaf ke ROOT — lalu ambil node tepat di bawah ROOT sebagai Level 2.

STDDEV_POP, bukan STDDEV_SAMP

Soal secara eksplisit meminta standard deviation populasi (STDDEV_POP), bukan sample. Ini memengaruhi nilai z-score dan batas threshold outlier.

UNION ALL: Summary + Detail dalam 1 Output

Karena hanya boleh satu output, digunakan UNION ALL untuk menggabungkan baris ringkasan (jumlah_anomali filled, id NULL) dan baris detail (id filled, jumlah_anomali NULL).

Format Output

Struktur hasil query final

level2	jumlah_anomali	id	nilai_order	average	stdev	jarak_average	z_score
N0601	2	NULL	NULL	NULL	NULL	NULL	NULL
N0601	NULL	N0123	7.200.000	4.000.000	800.000	3.200.000	4.00
N0601	NULL	N0456	1.300.000	4.000.000	800.000	-2.700.000	-3.38



level2 = Sales Manager Level 2



nilai_order tinggi (outlier atas)



nilai_order rendah (outlier bawah)



z_score ($|z| > 3$ = outlier)

Summary row: jumlah_anomali diisi, kolom detail = NULL | Detail row: kolom detail diisi, jumlah_anomali = NULL

Key Takeaways

Yang bisa dibawa ke use case nyata



Tree Traversal tanpa Rekursi

Self-JOIN berlapis adalah alternatif kuat untuk menelusuri hierarki di MySQL 5.x, meski lebih verbose dari CTE rekursif di MySQL 8.



Statistik dalam SQL

AVG, STDDEV_POP, dan Z-score bisa dihitung murni di SQL — tidak perlu membawa data ke Python atau Excel untuk analisis dasar.



Single Output, Dual Info

UNION ALL memungkinkan satu query menghasilkan format laporan yang kaya: summary dan detail digabung dengan NULL sebagai pemisah semantik.

Let's Connect!



miftahulfarohi25@gmail.com



[linkedin.com/in/ahmad-miftahul-farohi](https://www.linkedin.com/in/ahmad-miftahul-farohi)